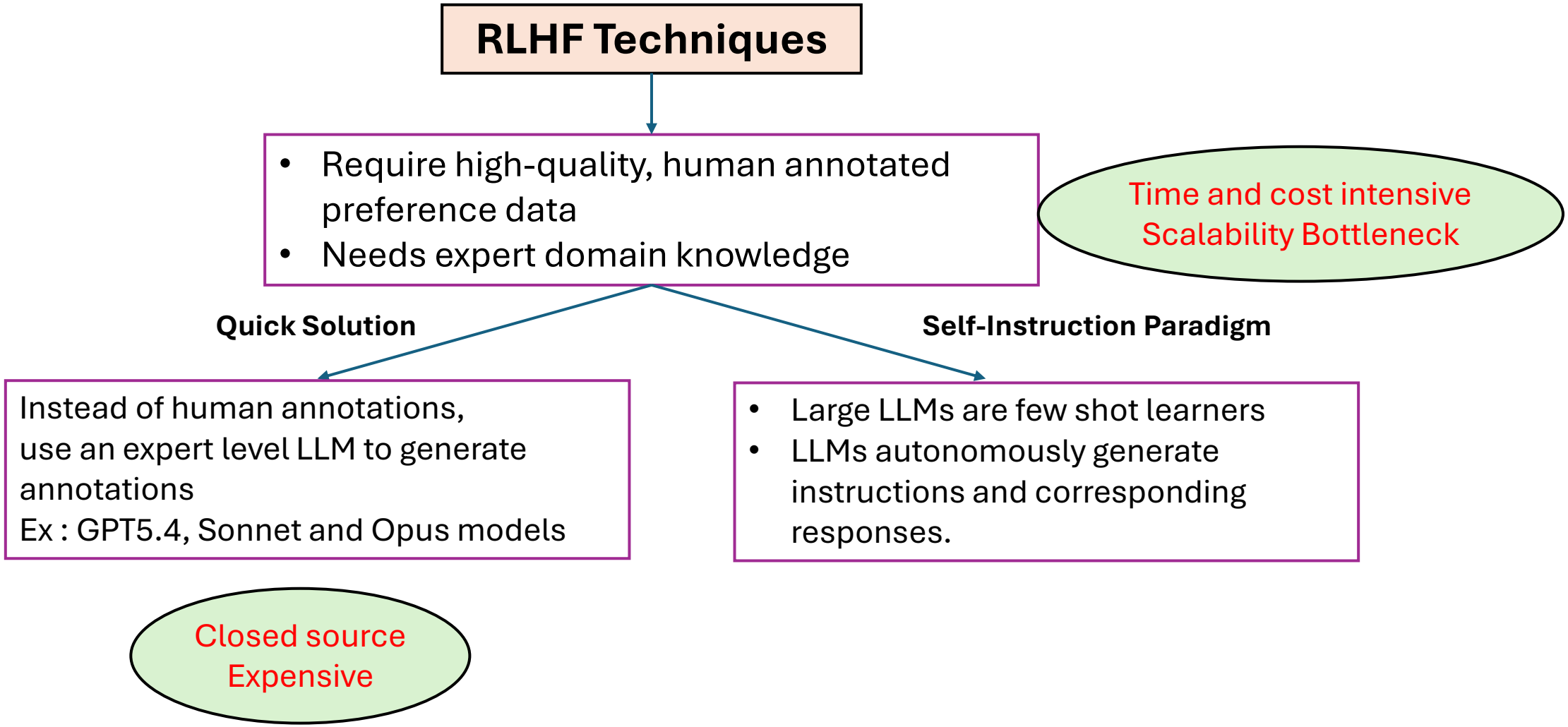
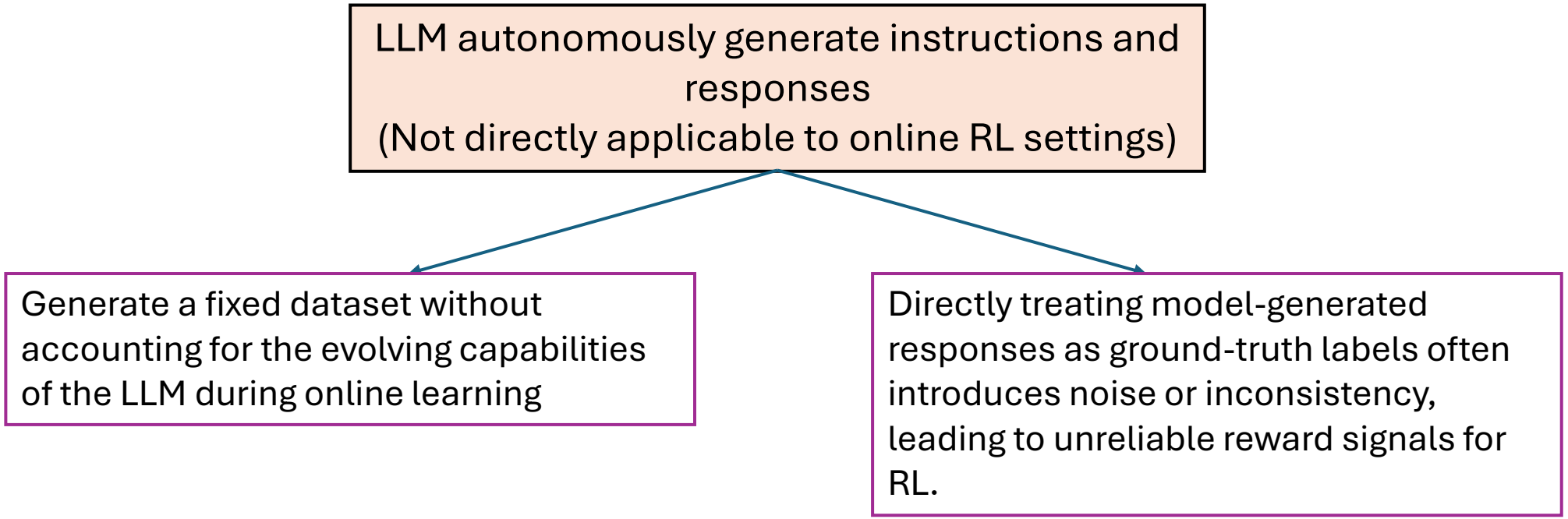
Self Play Reinforcement Learning in LLMs

Vrishab
Himanshu
Ashwin
Saksham

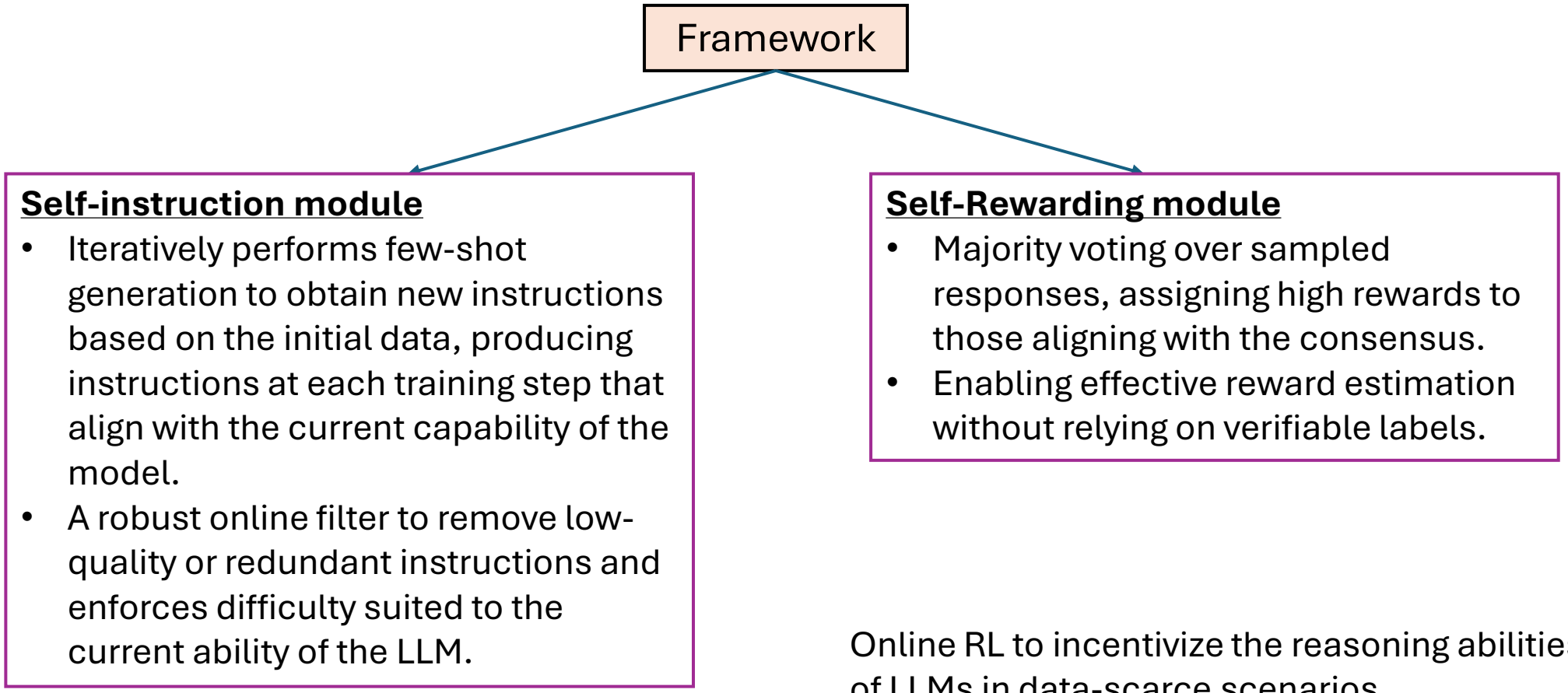
Setting Up the Background



Self-Instruct



Self-play Reinforcement Learning (SeRL)



Online RL to incentivize the reasoning abilities of LLMs in data-scarce scenarios

Methodology

```
for i in range N iterations:
        while total_generated_instructions >= threshold :
            generate instructions (Instruction set : B_T)
            sample n_{Vote} responses for each of the instructions
            Find the majority voting answer
            Apply the filter
            Based on this evaluate all the n_{Vote} answer
            Calculate advantage for each n_{Vote} answer
            Update the parameters (using PPO objective)
            Update the task pool
```

Self-instruction

Self-rewarding

Hyperparameters used:

- **N =** 3 iterations
- **Threshold =** 7500 instructions
- **|B_T| =** 16 new instructions
- **N_{Vote} =** 16
- **RL algorithm :** Reinforce++
- **LLM Backbone :** LLaMA-3.2-3B-Instruct , Qwen-2.5-7B-Instruct

500 seed tasks (1 instruction + 1 instance per task)

Instruction: Find the lcm of 5 and 13
Output: 65

LLM

Step1 : Instruction Generation

Please act as a professional math teacher. Your goal is to create high quality math word problems to help students learn math. You only need to create the new question. Please DO NOT solve it. Come up with a series of tasks:
Task 1: {instruction for existing task 1}
Task 2: {instruction for existing task 2}
…
Task 8: {instruction for existing task 8}
Task 9:

16 new instructions

Filtering

- Exclude instructions containing specific keywords like graph, image etc that can't be processed by LM
- Very long (<150 words) and short (<3) instructions are discarded
- If $max \begin{pmatrix} Rouge\ L(new, I1) \\ \cdots \\ Rouge\ L(new, I8) \end{pmatrix} \leq 0.7$, keep

Methodology

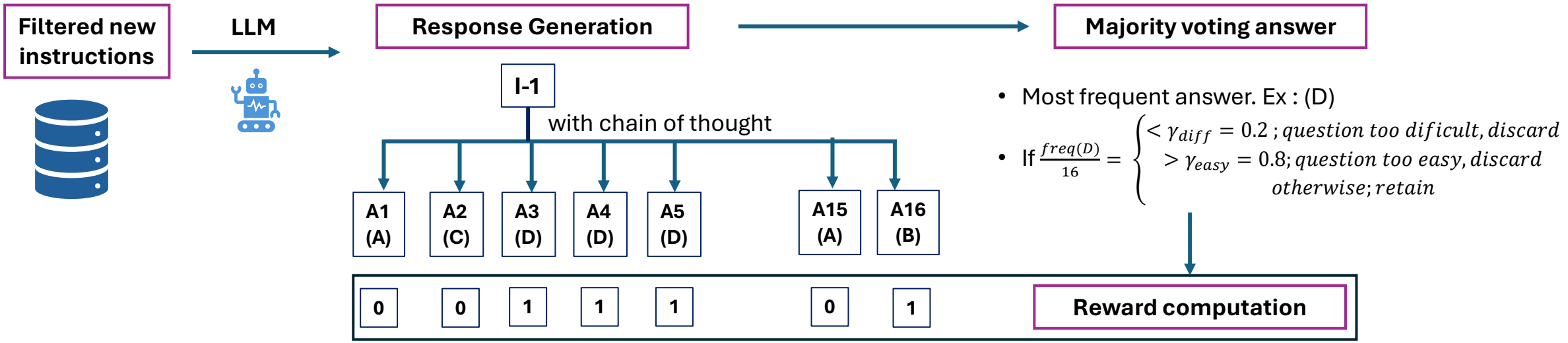
```
for i in range N iterations:
        while total_generated_instructions >= threshold :
            generate instructions (Instruction set : B_T)
            sample n_{Vote} responses for each of the instructions
            Find the majority voting answer
            Apply the filter
            Based on this evaluate all the n_{Vote} answer
            Calculate advantage for each n_{Vote} answer
            Update the parameters (using PPO objective)
            Update the task pool
```

Self-instruction

Self-rewarding

Hyperparameters used:

- **N =** 3 iterations
- **Threshold =** 7500 instructions
- **|B_T| =** 16 new instructions
- **N_{Vote} =** 16
- **RL algorithm :** Reinforce++
- **LLM Backbone :** LLaMA-3.2-3B-Instruct , Qwen-2.5-7B-Instruct



- Most frequent answer. Ex : (D)
- If $\frac{freq(D)}{16} = \begin{cases} < \gamma_{diff} = 0.2 ; question\ too\ dificult, discard \\ > \gamma_{easy} = 0.8; question\ too\ easy, discard \\ otherwise; retain \end{cases}$

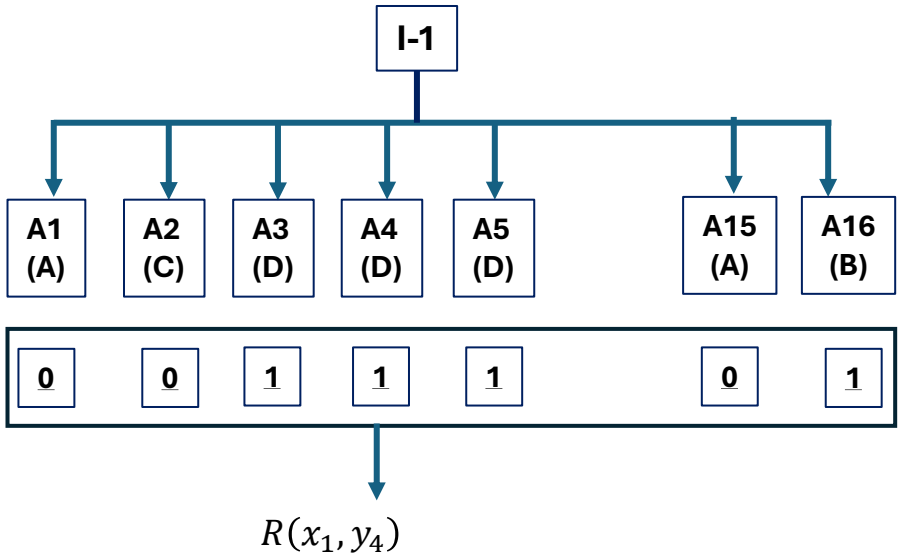
For subjective answers, chain of thought reasoning and answer the score is given by same LLM (LLM as a judge)

Methodology

```
for i in range N iterations:
        while total_generated_instructions >= threshold :
            generate instructions (Instruction set : B_T)
            sample n_{Vote} responses for each of the instructions
            Find the majority voting answer
            Apply the filter
            Based on this evaluate all the n_{Vote} answer
            Calculate advantage for each n_{Vote} answer
            Update the parameters (using PPO objective)
            Update the task pool
```

Self-instruction

Self-rewarding



Reinforce++ (Critic-free)

$$G(x_i, y_j) = R(x_i, y_j) - \sum_{k=t}^{T} KL_{ij}(k)$$

Suppose x_i has t tokens and y_j has T-t tokens

$$KL_{ij}(k) = -log\left[\frac{\pi_\theta(y_k|x_i,\ y_{<k})}{\pi_{ref}(y_k|x_i,\ y_{<k})}\right]$$

Suppose we have n_{bs} instructions at step t

$$\mu_t = \frac{1}{n_{bs}n_{vote}}\sum_{i=1}^{n_{bs}}\sum_{j=1}^{n_{vote}} G(x_i, y_j), \sigma_t = \sqrt{\frac{1}{n_{bs}n_{vote}}\sum_{i=1}^{n_{bs}}\sum_{j=1}^{n_{vote}}\left(G(x_i, y_j) - \mu_t\right)^2}$$

$$\text{Advantage, } \mathrm{A}(\mathrm{x_i}, \mathrm{y_j}) = \frac{G(x_i, y_j) - \mu_t}{\sigma_t}$$

Methodology

```
for i in range N iterations:
        while total_generated_instructions >= threshold :
            generate instructions (Instruction set : B_T)
            sample n_{Vote} responses for each of the instructions
            Find the majority voting answer
            Apply the filter
            Based on this evaluate all the n_{Vote} answer
            Calculate advantage for each n_{Vote} answer
            Update the parameters (using PPO objective)
            Update the task pool
```

Self-instruction

Self-rewarding

Now we have advantage for each question and answer

Advantage, $\mathrm{A}(\mathrm{x_i}, \mathrm{y_j}) = \frac{G(x_i, y_j) - \mu_t}{\sigma_t}$

PPO objective (n_{bs} questions in each step):

$$\mathcal{J}(\theta) = \frac{1}{n_{bs} n_{vote}} \sum_{i=1}^{n_{bs}} \sum_{j=1}^{n_{vote}} A(\mathrm{x_i}, \mathrm{y_j}) \frac{1}{|y|} \sum_{t=1}^{|y|} \min\left(\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\theta_{old}}(y_t | x, y_{<t})}, clip\left(\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\theta_{old}}(y_t | x, y_{<t})}, 1 - \epsilon, 1 + \epsilon \right) \right)$$

Where $\pi_{\theta_{old}}$ is the model parameter before the most recent model update

Results

Table 1: Pass@1 comparison across benchmarks between our method and baseline approaches, with all models evaluated using greedy decoding. Initial refers to the original LLaMA-3.2-3B-Instruct or Qwen-2.5-7B-Instruct model. **Bold** indicates the best performance.

Models	Iteration	Methods	MATH-500	MATH-Hard	ASDiv	College Math	TabMWP
LLaMA-3.2 -3B-Instruct		Initial	47.6	22.5	84.6	35.2	46.4
		RL-GT	49.7	**24.1**	88.9	36.4	69.9
	Iter1	SR-DPO	42.4	20.4	80.9	31.5	42.0
		I-RPO	44.0	17.8	86.1	32.2	58.9
		SeRL	48.6	23.0	87.5	36.7	68.4
	Iter2	SR-DPO	38.3	19.4	61.8	27.9	34.1
		I-RPO	42.9	18.6	87.8	31.1	58.2
		SeRL	50.4	23.6	88.9	**38.2**	**72.3**
	Iter3	SR-DPO	32.7	17.6	57.5	23.6	29.5
		I-RPO	42.5	18.1	87.6	27.9	52.2
		SeRL	**52.6**	23.7	**89.0**	37.7	70.6
Qwen-2.5 -7B-Instruct		Initial	74.2	48.8	93.5	54.3	75.5
		RL-GT	74.8	**50.9**	94.3	**55.5**	72.7
	Iter1	SR-DPO	72.6	49.2	94.0	54.6	**80.7**
		I-RPO	71.4	47.6	91.4	53.8	71.6
		SeRL	74.2	50.0	94.2	54.3	77.0
	Iter2	SR-DPO	73.8	50.0	94.1	55.0	78.7
		I-RPO	74.0	45.2	93.9	53.2	73.7
		SeRL	74.8	49.7	**94.7**	55.1	80.1
	Iter3	SR-DPO	71.6	47.4	92.4	53.0	72.7
		I-RPO	72.8	46.3	92.5	52.0	71.4
		SeRL	**75.8**	50.4	94.4	55.1	79.4

Iterative methods

I-RPO (Iterative Reasoning Preference Optimisation)

- Rule based reward is used to create preferential data (extracting the answer from the response using regular expressions and comparing it against the ground-truth answer to assign a reward)

SR-DPO (Self Rewarding DPO)

- Model based reward (LLM as a judge)

Observations

- SR-DPO and I-RPO exhibits model degradation over multiple iterations
- SR-DPO performs better on Qwen 7B model compared to LLaMA 3B (suggests that rewards estimates are accurate for larger models)
- SeRL outperforms across all iterations

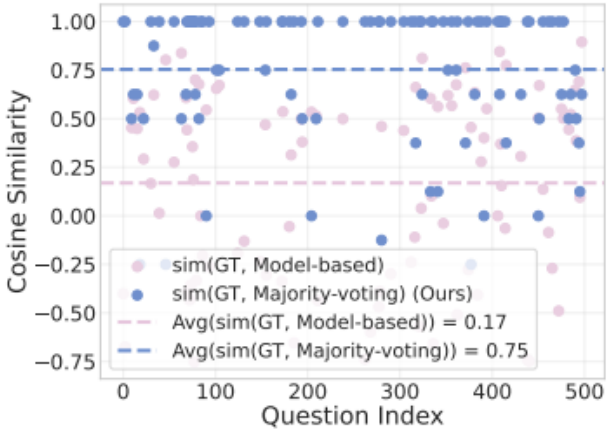
Results

Table 3: Pass@1 comparison under the same number of training steps with different filtering strategies ablated, using LLaMA-3.2-3B-Instruct.

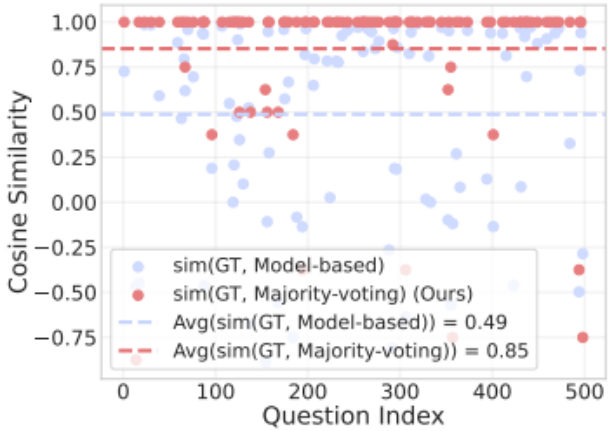
Methods	MATH-500	MATH-Hard	ASDiv	College Math	TabMWP
SeRL	52.6	23.7	89.0	37.7	70.6
SeRL w/o Length Filter	47.6	23.2	87.4	36.2	60.1
SeRL w/o Keywords Filter	48.0	22.1	87.4	36.3	62.5
SeRL w/o Similarity Filter	48.8	23.3	87.3	35.6	64.6
SeRL w/o Difficulty Filter	11.6	5.1	1.5	14.0	10.2

Observations

- Filtering strategy is effective



(a) The result of LLaMA-3.2-3B-Instruct.



(b) The result of Qwen-2.5-7B-Instruct.

Observations

- Majority voting reward aligns with the ground truth reward
- Model based rewards aligns better with increase in LLM parameters

Results

Table 5: Comparison of SeRL performance under different reinforcement learning algorithms. **Bold** indicates the best performance.

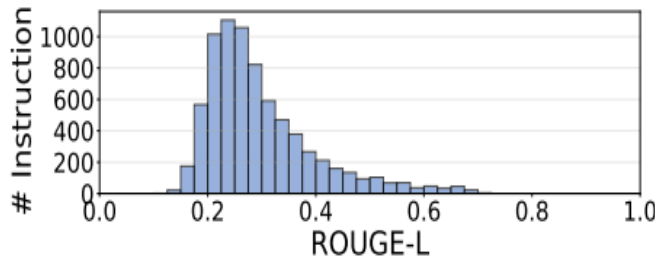
Models	Algorithm	Methods	MATH-500	MATH-Hard	ASDiv	College Math	TabMWP
LLaMA -3.2-3B -Instruct	RLOO	SeRL(iter1)	49.1	24.3	87.7	36.0	62.7
		SeRL(iter2)	49.2	**24.7**	88.9	37.7	70.1
		SeRL(iter3)	51.7	23.8	**89.0**	36.7	70.3
	GRPO	SeRL(iter1)	50.4	23.4	88.0	36.6	64.8
		SeRL(iter2)	49.2	23.2	88.4	36.0	67.3
		SeRL(iter3)	48.6	24.0	88.3	36.1	67.6
	Reinforce++	SeRL(iter1)	48.6	23.0	87.5	36.7	68.4
		SeRL(iter2)	50.4	23.6	88.9	**38.2**	**72.3**
		SeRL(iter3)	**52.6**	23.7	**89.0**	37.7	70.6

Observations

- SeRL is applicable to different RL algorithms
- Reinforce++ more robust compared to GRPO and RLOO

Observations

- Most generated instructions exhibit low ROUGE-L similarity with D_{seed}. This indicates that the model does not merely mimic D_{seed} but instead generates diverse and novel instructions.
- In UMAP, generated instruction points are not tightly clustered but instead exhibit a wide distribution similar to that of the MATH training set. This demonstrates the diversity of the generated dataset.



(d) Distribution of ROUGE-L scores between $\mathcal{D}^1_{gen}$ and D_{seed} after difficulty filtering.

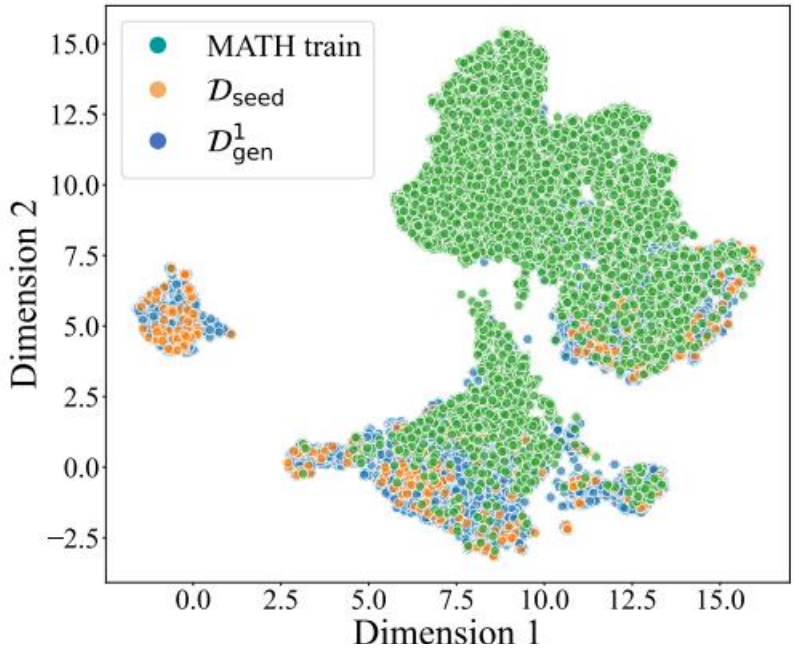
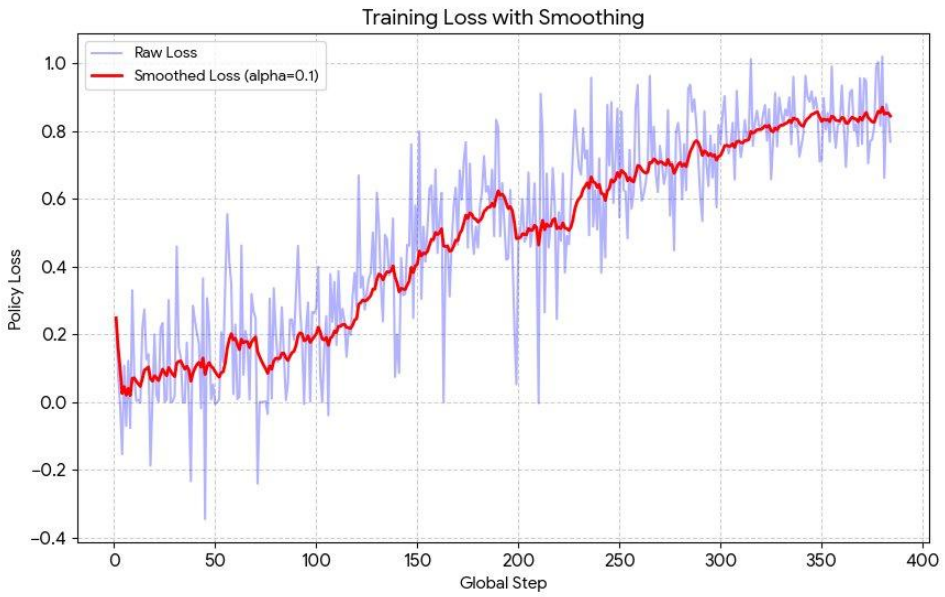
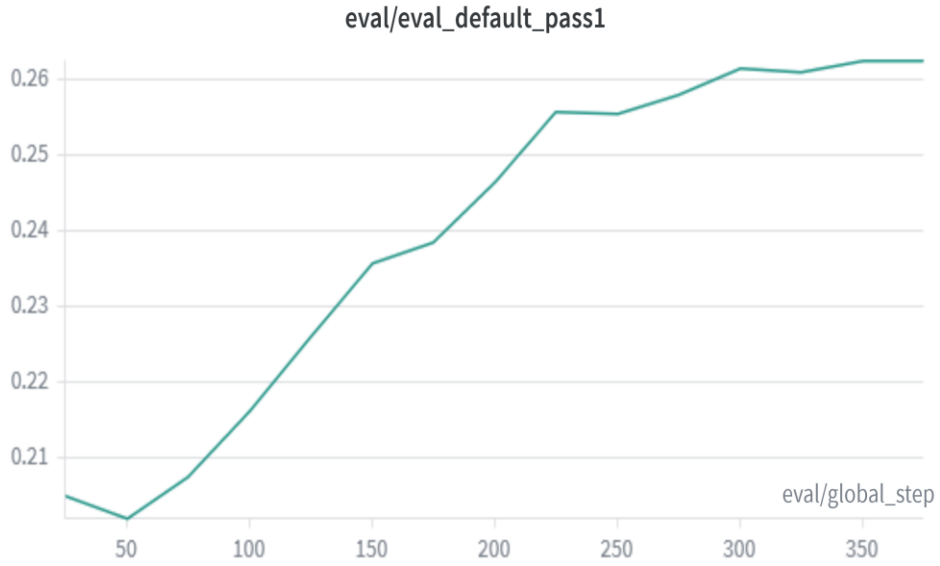


Figure 6: UMAP projection of BERT feature distributions for $\mathcal{D}^1_{gen}$, $\mathcal{D}_{seed}$, and MATH training set.

Results (from Experiments)



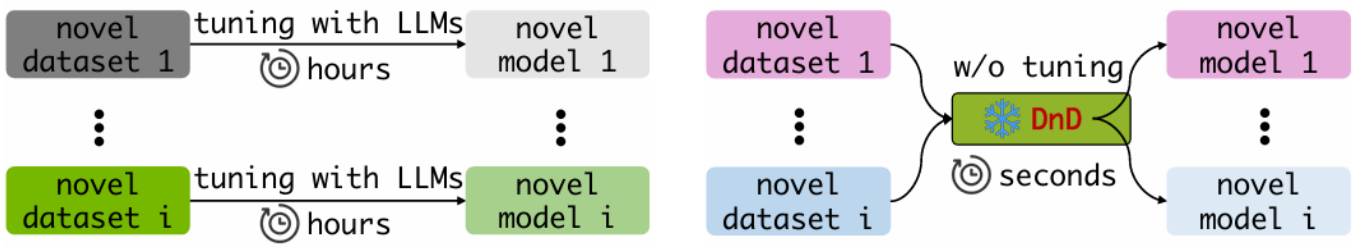


Our contribution

- Problem with current methods: Domain adaptation approaches for LLMs combine cross-task generalization and task-specific adaptation in a single objective
- This is causes the demands to conflict and cause sever gradient interference, causing the model to be trapped in generic compromise solutions and fail for task specificity
- We purpose enforcing 3 independent stages

Stage 1

- **Foundation consolidation:** First we find the generalized LoRA adapters using DnD hypernet



Stage 2

- **Knowledge Task Alignment:** Because the generated matrices may not perfectly match the new target domain, we perform explicit alignment step using a conditional VAE to map these generalized weights to perfect matching weights by conditioning on task's semantic
- We create 2 vectors , one from pretrained model and task aligned model , we subtract them and we get the direction in which we need to align our lora weights , so we use a conditional VAE, given the direction to align our LoRA weights. We are using VAE so we can have smooth loss (KL div), so we know that our VAE is actually aligning the weights in the task specific direction

Stage 3

Core Objective

- Executes lightweight, task-conditioned specialization on the semantically aligned parameters produced in Stage 2.
- Operates entirely at deployment/inference time, avoiding prolonged online fine-tuning.
- **The Mechanism: Adaptation as Decision-Making**
- Rather than applying a fixed gradient rule, Stage 3 frames refinement as a decision-making problem.
- An RL-trained refinement policy evaluates the unseen target task and autonomously selects the optimal strategy from a predefined pool.
- **The Strategy Arsenal (Action Space)**
- **Test-Time Learning (TTL)**
- **Two-Subspace Mixing LoRA**
- **Test-Time Scaling (TTS)**
- **Latent Space Modification (SLOT)**

Results

Dataset	SeRL	ID
ARC-c	22.44	73.7
ARC-e	23.99	88.4
HellaSwag	25.04	57.2
BoolQ	44.74	58.8
WinoGrande	49.57	57.3

Dataset	SeRL	OOD
GSM-8k	23.96	30.3
MATH	24.66	24.5
DivLogicEval	27.44	28.7
SocialIQA	32.96	55.1
CodeMMLU	26.26	35.6
JAMA	12.36	31.2

In Domain: We are evaluating on the test split and the train split is included in the training dataset for stage 1
Out of Domain: The model sees the test data at the time of evaluation only

Initial Seed data size : 20 questions

Model used : Qwen 2.5 0.5B instruct

Compute : 298 GPU hours

References

1. Fang, Wenkai, Shunyu Liu, Yang Zhou, Kongcheng Zhang, Tongya Zheng, Kaixuan Chen, Mingli Song, and Dacheng Tao. "Serl: Self-play reinforcement learning for large language models with limited data." *arXiv preprint arXiv:2505.20347* (2025).
2. Liang, Zhiyuan, Dongwen Tang, Yuhao Zhou, Xuanlei Zhao, Mingjia Shi, Wangbo Zhao, Zekai Li et al. "Drag-and-drop llms: Zero-shot prompt-to-weights." *arXiv preprint arXiv:2506.16406* (2025).
3. Hu, Jian. "Reinforce++: A simple and efficient approach for aligning large language models." *arXiv e-prints* (2025): arXiv-2501.
4. Shao, Zhihong, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models." *arXiv preprint arXiv:2402.03300* (2024).
5. Wang, Yizhong, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. "Self-instruct: Aligning language models with self-generated instructions." In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pp. 13484-13508. 2023.

Appendix (Self-Instruct)

175 seed tasks
(1 instruction + 1 instance per task)

Instruction: A color description has been provided. Find the CSS code associated with that color.
Input: A light red color with a medium light shade of pink
Output: #DB7093

Task pool
Sampled 8 tasks in each step
(6 human + 2 LM)

Filtering

- If $\min(RL(new, i): 1 \leq i \leq 8) \leq 0.7$, keep
- Exclude instructions containing specific keywords like graph, image etc that can't be processed by LM
- Very long and short instructions are discarded

LLM

Step1 : Instruction Generation

Come up with a series of tasks:
Task 1: {instruction for existing task 1}
Task 2: {instruction for existing task 2}
…
Task 8: {instruction for existing task 8}
Task 9:

Step3 : Instance Generation

Instruction : Find out if the given text is in favor of or against abortion.
Class Label: Pro-abortion
Input: I believe that women should have the right to choose whether or not they want to have an abortion

Instruction : Give me a quote from a famous person on this topic.
Input: Topic: The importance of being honest.
Output: "Honesty is the first chapter in the book of wisdom." - Thomas Jefferson

Step2 : Classification Task Identification

Can the following task be regarded as a classification task with finite output labels?
Task: Given my personality and the job, tell me if I would be suitable. Is it classification? Yes
Task: Give me an example of a time when you had to use your sense of humor. Is it classification? No
…
Target Task

Yes

Output-first

No

Input-first